Minimum spanning trees: solutions

23-E01 Respecting cuts and quantifiers for ties

Derivation

The crossing edges are exactly cd3,be4,bd5,ce6. Edges ac,bc lie inside S and de inside its complement, so no A edge crosses. Choose an MST T containing A. If cd is absent, the cycle in T+cd has another crossing edge f∉A. Removing f leaves a tree, and $w(f) \geq 3$ makes the cost nonincreasing while preserving A. Thus cd is safe.

For triangle x,y,z with all weights 1, the MSTs {xy,yz},{xy,xz},{xz,yz} all cost 2. Each edge is light for some cut and occurs in two MSTs, so the first statement is true. Edge xy is absent from the third tree, refuting the second. All three edges together form a cycle, refuting the third.

If a uniquely heaviest cycle edge e belonged to an MST, removing e would define a cut. The rest of the cycle provides a crossing f with $w(f)<w(e)$. Exchanging preserves a tree and strictly lowers cost, contradicting optimality.

Conclusion and checks

Edge cd:3 is safe here. Some MST cannot be strengthened to every MST for a tied light edge. Unique lightness forces inclusion; unique cycle heaviness forces exclusion.

23-E02 All Kruskal acceptances and rejections

Derivation

Sorted order is bc1(ID2),ac2(ID1),de2(ID6),cd3(ID4),ab4(ID0),be4(ID7),bd5(ID3),ce6(ID5).

Edge	Endpoint components before scan	Decision and components afterward	Cost
bc1	{b},{c}	accept; {a},{bc},{d},{e}	1
ac2	{a},{bc}	accept; {abc},{d},{e}	3
de2	{d},{e}	accept; {abc},{de}	5
cd3	{abc},{de}	accept; {abcde}	8
ab4	{abcde},{abcde}	reject; unchanged	8
be4	same	reject; unchanged	8
bd5	same	reject; unchanged	8
ce6	same	reject; unchanged	8

Minimum spanning trees: solutions (continued)

For known connected input, stop at the fourth scan after n−1 accepted edges. With be1, order becomes bc1,be1,ac2,de2,cd3,ab4,bd5,ce6. Accept the first four, with growing components {bc},{bce},{abce},{abcde} and costs 1,2,4,6. Reject all others. Reordering equal-weight edges still selects a light edge for a respecting cut, so IDs do not determine optimality.

Conclusion and checks

Original cost is 8; after the decrease it is 6. Rejected endpoints belong to one **current A component**. Testing original graph components instead would reject every edge of a connected input.

23-E03 Costs under different graph models

Derivation

The full Kruskal bound is $O(m \log(m+1)+(n+m)(1+\alpha(n)))$. For $n=1000, \alpha(n) \geq 1$, giving $O(2000 \log 2001+3000\alpha(1000))$ for the simple case and $O(10^7 \log(10^7+1)+(10^7+1000)\alpha(1000))$ for the multigraph. Parallel edges permit m far beyond n^2, so the simple-graph argument for $\log m=O(\log n)$ is unavailable. This concerns asymptotic assumptions; two fixed numerical inputs alone cannot prove a functional relationship.

Binary-heap Prim gives respectively $O(3000 \log 1000)$ and $O((10^7+1000)\log 1000)$. Fibonacci Prim gives $O(2000+1000 \log 1000)$ and $O(10^7+1000 \log 1000)$. Linear-key search with the original lists gives $O(10^6+2000)$ and $O(10^6+10^7)$. Building a matrix that retains only the lightest parallel edge still requires $\Omega(m)$ input work before $\Theta(n^2)$ matrix Prim.

Counting sort takes $O(m+10)$; add DSU $O((n+m)\alpha(n))$ here and input reading. Avoiding cycles proves only extension to some tree. Optimal extension additionally needs the accepted edge to be light across a current component cut, followed by the exchange theorem.

Conclusion and checks

The three queue contracts give different bounds. A binary implementation cannot claim Fibonacci cost. Comparison algorithms tolerate negative weights; counting sort here explicitly assumes 0..9.

Minimum spanning trees: solutions (continued)

23-E04 Competing Prim candidates

Derivation

Initially S=∅,key[a]=0 and all other keys∞. States after each scan are

Extraction and S	Unselected key(parent)	A and cost
a0; {a}	b4(a),c2(a),d∞,e∞	∅; 0
c2; {a,c}	b1(c),d3(c),e6(c)	ac; 2
b1; {a,b,c}	d3(c),e4(b)	ac,cb; 3
d3; {a,b,c,d}	e2(d)	ac,cb,cd; 6
e2; all vertices	none	ac,cb,cd,de; 8

Vertex b changes∞→4(parent a)→1(parent c). Vertex e changes∞→6(parent c)→4(parent b)→2(parent d). Both c2 then b1 and d3 then e2 decrease. Adding vertices changes the candidate cut, so no invariant requires monotone keys.

The update key[u]+w(u,v) accumulates distance from a root. On the triangle, after a, b4,c5 are candidates. Prim uses bc2 to change c to key2,parent b. The path algorithm compares4+2=6 with 5 and retains parent a. Tree costs are 6 and 9 respectively; their a-to-c path lengths are 6 and 5.

Conclusion and checks

The five-vertex Prim result costs 8, with parent[c]=a,parent[b]=c,parent[d]=c,parent[e]=d. A key is the weight of a crossing edge, not distance from a.

23-E05 Certificates, uniqueness, and weight decreases

Derivation

Nontree edge	T path and weights	M_T	w−M_T
ab4	a-c-b; 2,1	2	2
bd5	b-c-d; 1,3	3	2
ce6	c-d-e; 3,2	3	3
be4	b-c-d-e; 1,3,2	3	1

Minimum spanning trees: solutions (continued)

All comparisons are strict, so T is unique. Lowering be to 3 gives w′=M=3. Old T remains optimal at 8; replacing cd by be produces {bc,ac,de,be}, also at 8, so uniqueness fails. Lowering be to 1 gives the same exchanged tree at 6. Its nontree edge cd has path c-b-e-d with maximum2<3; ab has path a-c-b, maximum2<4; bd has path b-e-d, maximum2<5; ce has path c-b-e, maximum1<6. All are strict, so this new MST is unique.

Generally let W=w(T). A new tree R omitting e retains its old cost, at least W. If R contains e, removing e defines a cut. The original T path between its endpoints supplies a crossing g with w(g)≤M. Since R−e+g is an original spanning tree, W≤w(R)−w′+w(g)≤w(R)−w′+M. Thus w(R)≥W+w′−M. Combining both cases gives W+min(0,w′−M), attained by keeping T or replacing its maximum path edge.

Conclusion and checks

At 3 the optimum is 8 and not unique; at 1 it is 6 and unique. The assumption is a decrease of one original non-self-loop nontree edge. It does not cover increasing a tree edge or changing several edges together.

23-E06 Negative weights, parallel edges, and isolation

Derivation

Sorted edges are e0−100,e1−2,e3 0,e2 1,e4 4. Reject e0 since both endpoints are a. Accept e1 to join a,b at cost−2; accept e3 to include c at unchanged cost−2. Reject e2 and e4 because their endpoints are already connected. Output {e1,e3}, a tree on {a,b,c} and an empty tree on {d}.

Prim from a extracts a0,b−2,c0, then faces d∞. The connected version must report disconnected input at that point. The forest version makes d a new root with key0,parent NIL, and adds no edge. Infinity signifies no crossing edge. The negative self-loop, despite its weight, neither connects distinct vertices nor preserves acyclicity.

Conclusion and checks

The MSF costs−2 and has n−number of original components=4−2=2 edges. No whole-graph MST exists. The parallel ab edges have distinct IDs; e1 is selected without confusing it with e2.

Minimum spanning trees: solutions (continued)

23-E07 Deriving the cut certificate in reverse

Derivation

Quantify only over non-self-loop nontree edges. Self-loops never belong to a spanning tree, and an empty-path maximum is unnecessary. If an MST had $w(e)<M_T$, adding e and deleting a maximum cycle edge would strictly improve it. Conversely, for each $f \in T$, removing f defines a cut. Every other crossing e has a T path through f, hence $w(e) \geq M_T \geq w(f)$. Thus f is light. Any prefix assembled from T respects that cut, so repeated safe-edge exchanges establish optimal extendibility and ultimately optimality of T.

If some nontree comparison is equality, the corresponding exchange gives another equally cheap tree. If all are strict, every $T-f$ cut has f as its unique light edge, forcing f into every MST. All T edges are forced, so T is unique.

For $T'=\{ab1,ac2\}$, nontree bc1 has path b-a-c of maximum2, violating$1 \geq 2$. Replace ac2 by bc1 to obtain {ab,bc} at cost 2. Input weights 1,1,2 repeat, but the remaining nontree ac2 has path maximum1 and satisfies$2>1$, so the MST is unique.

Conclusion and checks

First verify the candidate is a spanning tree, or M_T may not have a unique defining path. Distinct input weights suffice for uniqueness but are not necessary; the path certificate is the exact criterion.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Minimum spanning trees: solutions (continued)

Verification code appendix: kruskal_mst.py

```
"""Kruskal MST with a compact disjoint-set implementation."""

def mst(vertices, edges):
    parent = {v: v for v in vertices}
    rank = {v: 0 for v in vertices}

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    def union(a, b):
        ra, rb = find(a), find(b)
        if ra == rb:
            return False
        if rank[ra] < rank[rb]:
            ra, rb = rb, ra
        parent[rb] = ra
        if rank[ra] == rank[rb]:
            rank[ra] += 1
        return True

    chosen = []
    for weight, u, v in sorted(edges):
        if union(u, v):
            chosen.append((weight, u, v))
    return chosen


edges = [(2, "a", "c"), (5, "a", "d"), (1, "b", "c"), (4, "b", "d")]
chosen = mst({"a", "b", "c", "d"}, edges)
assert len(chosen) == 3 and sum(edge[0] for edge in chosen) == 7
print(chosen)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Minimum spanning trees: solutions (continued)

Verification code appendix: mst_checks.py

```
"""Compare Kruskal with exhaustive spanning trees on small graphs."""

from itertools import combinations
import random


class DisjointSet:
    def __init__(self, vertices):
        self.parent = {vertex: vertex for vertex in vertices}

    def find(self, item):
        while self.parent[item] != item:
            self.parent[item] = self.parent[self.parent[item]]
            item = self.parent[item]
        return item

    def union(self, left, right):
        left, right = self.find(left), self.find(right)
        if left == right:
            return False
        self.parent[left] = right
        return True


def kruskal(vertices, edges):
    result = []
    sets = DisjointSet(vertices)
    for weight, u, v in sorted(edges):
        if sets.union(u, v):
            result.append((weight, u, v))
    return result


def is_spanning_tree(vertices, edges):
    if len(edges) != len(vertices) - 1:
        return False
    sets = DisjointSet(vertices)
    return all(sets.union(u, v) for _, u, v in edges)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Minimum spanning trees: solutions (continued)

```
def brute_force_mst(vertices, edges):
    candidates = [
        selected
        for selected in combinations(edges, len(vertices) - 1)
        if is_spanning_tree(vertices, selected)
    ]
    return min((sum(edge[0] for edge in tree) for tree in candidates), default=None)


def run():
    triangle = [(1, "a", "b"), (3, "a", "c"), (2, "b", "c")]
    result = kruskal(["a", "b", "c"], triangle)
    assert sum(edge[0] for edge in result) == 3
    assert brute_force_mst(["a", "b", "c"], triangle) == 3

    rng = random.Random(23)
    trials = 0
    for size in range(3, 8):
        vertices = [f"v{i}" for i in range(size)]
        for _ in range(15):
            edges = []
            for i in range(size):
                for j in range(i + 1, size):
                    if rng.random() < 0.45:
                        edges.append((rng.randint(-3, 8), vertices[i], vertices[j]))
            if brute_force_mst(vertices, edges) is None:
                continue
            tree = kruskal(vertices, edges)
            assert is_spanning_tree(vertices, tree)
            assert sum(edge[0] for edge in tree) == brute_force_mst(vertices, edges)
            trials += 1
    assert trials > 20
    print(f"PASS triangle_MST=3 brute_force_cases={trials} negative_weights=allowed")


if __name__ == "__main__":
    run()
```

Minimum spanning trees: solutions (continued)

Verification code appendix: mst_depth_checks.py

```
"""MST/MSF behavior and certificates versus exhaustive spanning-forest subsets."""
from itertools import combinations,product
from math import inf
import random

SAMPLE=[(0,1,4),(0,2,2),(1,2,1),(1,3,5),(2,3,3),(2,4,6),(3,4,2),(1,4,4)]


class DSU:
    def __init__(self,n):self.parent=list(range(n));self.rank=[0]*n
    def find(self,x):
        if x!=self.parent[x]:self.parent[x]=self.find(self.parent[x])
        return self.parent[x]
    def union(self,a,b):
        a,b=self.find(a),self.find(b)
        if a==b:return False
        if self.rank[a]<self.rank[b]:a,b=b,a
        self.parent[b]=a
        if self.rank[a]==self.rank[b]:self.rank[a]+=1
        return True
    def groups(self):return frozenset(frozenset(i for i in range(len(self.parent)) if
self.find(i)==root) for root in set(self.find(i) for i in range(len(self.parent))))


def kruskal(n,edges):
    d=DSU(n);selected=[];trace=[]
    for identity in sorted(range(len(edges)),key=lambda i:(edges[i][2],i)):
        u,v,w=edges[identity];accepted=d.union(u,v)
        if accepted:selected.append(identity)
        trace.append((identity,accepted,d.groups()))
    return selected,trace


def prim(n,edges):
    # Linear minimum selection implements the documented dense variant, not a heap bound.
    adjacency=[[] for _ in range(n)]
    for identity,(u,v,w) in enumerate(edges):
        adjacency[u].append((v,w,identity));adjacency[v].append((u,w,identity))
    key=[inf]*n;parent=[None]*n;remaining=set(range(n));selected=[];trace=[]
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Minimum spanning trees: solutions (continued)

```
    if n:key[0]=0
    while remaining:
        u=min(remaining,key=lambda v:(key[v],v))
        if key[u]==inf:key[u]=0  # Explicitly start another connected component.
        extracted=key[u];remaining.remove(u)
        if parent[u] is not None:selected.append(parent[u])
        for v,w,identity in adjacency[u]:
            if v in remaining and w<key[v]:key[v]=w;parent[v]=identity
        trace.append((u,extracted,key[:],parent[:]))
    return selected,trace


def forest(n,edges,selected):
    d=DSU(n)
    for identity in selected:
        u,v,w=edges[identity]
        if not d.union(u,v):return None
    return d.groups()


def path_max(n,edges,selected,source,target):
    if source==target:return -inf
    adjacency=[[] for _ in range(n)]
    for identity in selected:
        u,v,w=edges[identity];adjacency[u].append((v,w));adjacency[v].append((u,w))
    pending=[(source,None,-inf)]
    while pending:
        u,parent,maximum=pending.pop()
        if u==target:return maximum
        for v,w in adjacency[u]:
            if v!=parent:pending.append((v,u,max(maximum,w)))
    return inf


def certificate(n,edges,selected,strict=False):
    allsets=DSU(n)
    for u,v,w in edges:allsets.union(u,v)
    if forest(n,edges,selected)!=allsets.groups():return False
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Minimum spanning trees: solutions (continued)

```
    chosen=set(selected)
    for identity,(u,v,w) in enumerate(edges):
        if identity in chosen or u==v:continue
        maximum=path_max(n,edges,selected,u,v)
        if w<maximum or (strict and w==maximum):return False
    return True


def oracle(n,edges):
    target=DSU(n)
    for u,v,w in edges:target.union(u,v)
    partition=target.groups();needed=n-len(partition);best=inf;optimal=[];candidates=[]
    for chosen in combinations(range(len(edges)),needed):
        if forest(n,edges,chosen)!=partition:continue
        value=sum(edges[i][2] for i in chosen);candidates.append((chosen,value))
        if value<best:best=value;optimal=[frozenset(chosen)]
        elif value==best:optimal.append(frozenset(chosen))
    return best,optimal,candidates


def verify(n,edges):
    best,optimal,candidates=oracle(n,edges)
    for chosen,_ in [kruskal(n,edges),prim(n,edges)]:
        assert frozenset(chosen) in optimal and sum(edges[i][2] for i in chosen)==best
        assert certificate(n,edges,chosen)
        assert certificate(n,edges,chosen,True)==(len(optimal)==1)
    for chosen,value in candidates:assert certificate(n,edges,chosen)==(value==best)
    # A unique light edge across a cut belongs to every optimum, not merely some.
    for mask in range(1,(1<<n)-1):
        crossing=[i for i,(u,v,w) in enumerate(edges) if ((mask>>u)&1)!=((mask>>v)&1)]
        if not crossing:continue
        minimum=min(edges[i][2] for i in crossing)
        light=[i for i in crossing if edges[i][2]==minimum]
        if len(light)==1:assert all(light[0] in chosen for chosen in optimal)
    return len(candidates)


def main():
```

Minimum spanning trees: solutions (continued)

```
    graphs=0;candidates=0
    for n in range(5):
        pairs=list(combinations(range(n),2))
        for weights in product([None,-1,0,2],repeat=len(pairs)):
            edges=[(u,v,w) for (u,v),w in zip(pairs,weights) if w is not None]
            candidates+=verify(n,edges);graphs+=1
    rng=random.Random(23)
    for _ in range(150):
        n=rng.randrange(2,7);pairs=list(combinations(range(n),2));rng.shuffle(pairs)
        edges=[(u,v,rng.randrange(-4,6)) for u,v in pairs[:min(9,len(pairs))]]
        candidates+=verify(n,edges);graphs+=1
    assert oracle(5,SAMPLE)[0]==8
    chosen,trace=kruskal(5,SAMPLE);assert chosen==[2,1,6,4]
    pt,ptrace=prim(5,SAMPLE);assert pt==[1,2,4,6]
    assert [(u,key) for u,key,*_ in ptrace]==[(0,0),(2,2),(1,1),(3,3),(4,2)]
    triangle=[(0,1,1),(1,2,1),(0,2,1)];verify(3,triangle)
    assert len(oracle(3,triangle)[1])==3
    multigraph=[(0,0,-100),(0,1,2),(0,1,-1),(1,2,0),(0,2,3),(3,4,-2)]
    verify(6,multigraph)
    assert sum(multigraph[i][2] for i in kruskal(6,multigraph)[0])==-3
    # Sensitivity: decrease outside edge; swap its cycle's maximum iff now cheaper.
    for identity,(u,v,w) in enumerate(SAMPLE):
        if identity in chosen:continue
        for reduced in range(-2,w+1):
            changed=SAMPLE[:];changed[identity]=(u,v,reduced)
            maximum=path_max(5,SAMPLE,chosen,u,v)
            best,opts,_=oracle(5,changed)
            assert best==8+min(0,reduced-maximum)
    print(f'PASS weighted_graphs={graphs} spanning_forest_candidates={candidates} '
          'Kruskal_and_dense_Prim=exhaustive_oracle certificate_iff_optimal=True '
          'unique_iff_strict_path_max=True cuts_unique_light_mandatory=True '
          'negative_parallel_selfloop_disconnected=True sample_weight=8 Prim_keys=0,2,1,3,2')


if __name__=='__main__':main()
```

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study